

MORESTLC

MORE ON THE SIMPLY TYPED LAMBDA-CALCULUS

```
Set Warnings "-notation-overridden,-parsing".
From PLF Require Import Maps.
From PLF Require Import Types.
From PLF Require Import Smallstep.
From PLF Require Import Stlc.
```

Simple Extensions to STLC

The simply typed lambda-calculus has a rich enough structure to make its theoretical properties interesting, but it is not much of a programming language!

In this chapter, we begin to close the gap with real-world languages by introducing a number of familiar features that have straightforward treatments at the level of typing.

Numbers

As we saw in the `STLCarith` exercises at the end of the `StlcProp` chapter, adding types, constants, and primitive operations for natural numbers is easy -- basically just a matter of combining the `Types` and `Stlc` chapters. Adding more realistic numeric types like machine integers and floats is also straightforward, though of course the specifications of the numeric primitives become more fiddly.

Let Bindings

When writing a complex expression, it is useful to be able to give names to some of its subexpressions to avoid repetition and increase readability. Most languages provide one or more ways of doing this. In OCaml (and Rocq), for example, we can write `let x=t1 in t2` to mean "reduce the expression `t1` to a value and bind the name `x` to this value while reducing `t2`."

Our `let`-binder follows OCaml in choosing a standard *call-by-value* evaluation order, where the `let`-bound term must be fully reduced before reduction of the `let`-body can begin. The typing rule `T_Let` tells us that the type of a `let` can be calculated by calculating the type of the `let`-bound term, extending the context with a binding with this type, and in this enriched context calculating the type of the body (which is then the type of the whole `let` expression).

At this point in the book, it's probably easier simply to look at the rules defining this new feature than to wade through a lot of English text conveying the same information. Here they are:

Syntax:

$$t ::=$$

$$| \dots$$

Terms

(other terms same as before)

| let $x=t_1$ in t_2 let-binding

Reduction:

$$\frac{t_1 \rightarrow t_1'}{\text{let } x=t_1 \text{ in } t_2 \rightarrow \text{let } x=t_1' \text{ in } t_2} \quad (\text{ST_Let1})$$

$$\frac{}{\text{let } x=v_1 \text{ in } t_2 \rightarrow [x:=v_1]t_2} \quad (\text{ST_LetValue})$$

Typing:

$$\frac{\Gamma \vdash t_1 \in T_1 \quad x \vdash T_1; \Gamma \vdash t_2 \in T_2}{\Gamma \vdash \text{let } x=t_1 \text{ in } t_2 \in T_2} \quad (\text{T_Let})$$

Pairs

Our functional programming examples in Rocq have made frequent use of *pairs* of values. The type of such a pair is called a *product type*.

The formalization of pairs is almost too simple to be worth discussing. However, let's look briefly at the various parts of the definition to emphasize the common pattern.

In Rocq, the primitive way of extracting the components of a pair is *pattern matching*. An alternative is to take `fst` and `snd` -- the first- and second-projection operators -- as primitives. Just for fun, let's do our pairs this way. For example, here's how we'd write a function that takes a pair of numbers and returns the pair of their sum and difference:

```
\x : Nat*Nat,
  let sum = x.fst + x.snd in
  let diff = x.fst - x.snd in
  (sum,diff)
```

Adding pairs to the simply typed lambda-calculus, then, involves adding two new forms of term -- pairing, written (t_1, t_2) , and projection, written $t.fst$ for the first projection from t and $t.snd$ for the second projection -- plus one new type constructor, $T_1 \times T_2$, called the *product* of T_1 and T_2 .

Syntax:

$t ::=$	Terms
...	
(t_1, t_2)	pair
$t_0.fst$	first projection
$t_0.snd$	second projection
$v ::=$	Values
...	
(v_1, v_2)	pair value
$T ::=$	Types
...	
$T_1 * T_2$	product type

For reduction, we need several new rules specifying how pairs and projection behave.

$$\frac{t_1 \rightarrow t_1'}{(t_1, t_2) \rightarrow (t_1', t_2)} \quad (\text{ST_Pair1})$$

$$\frac{t_2 \rightarrow t_2'}{(v_1, t_2) \rightarrow (v_1, t_2')} \quad (\text{ST_Pair2})$$

$$\frac{t_0 \rightarrow t_0'}{t_0.\text{fst} \rightarrow t_0'.\text{fst}} \quad (\text{ST_Fst1})$$

$$\frac{}{(v_1, v_2).\text{fst} \rightarrow v_1} \quad (\text{ST_FstPair})$$

$$\frac{t_0 \rightarrow t_0'}{t_0.\text{snd} \rightarrow t_0'.\text{snd}} \quad (\text{ST_Snd1})$$

$$\frac{}{(v_1, v_2).\text{snd} \rightarrow v_2} \quad (\text{ST_SndPair})$$

Rules `ST_FstPair` and `ST_SndPair` say that, when a fully reduced pair meets a first or second projection, the result is the appropriate component. The congruence rules `ST_Fst1` and `ST_Snd1` allow reduction to proceed under projections, when the term being projected from has not yet been fully reduced. `ST_Pair1` and `ST_Pair2` reduce the parts of pairs: first the left part, and then -- when a value appears on the left -- the right part. The ordering arising from the use of the metavariables v and t in these rules enforces a left-to-right evaluation strategy for pairs. (Note the implicit convention that metavariables like v and v_1 can only denote values.) We've also added a clause to the definition of values, above, specifying that (v_1, v_2) is a value. The fact that the components of a pair value must themselves be values ensures that a pair passed as an argument to a function will be fully reduced before the function body starts executing.

The typing rules for pairs and projections are straightforward.

$$\frac{\Gamma \vdash t_1 \in T_1 \quad \Gamma \vdash t_2 \in T_2}{\Gamma \vdash (t_1, t_2) \in T_1 * T_2} \quad (\text{T_Pair})$$

$$\frac{\Gamma \vdash t_0 \in T_1 * T_2}{\Gamma \vdash t_0.\text{fst} \in T_1} \quad (\text{T_Fst})$$

$$\frac{\Gamma \vdash t_0 \in T_1 * T_2}{\Gamma \vdash t_0.\text{snd} \in T_2} \quad (\text{T_Snd})$$

`T_Pair` says that (t_1, t_2) has type $T_1 * T_2$ if t_1 has type T_1 and t_2 has type T_2 .

Conversely, `T_Fst` and `T_Snd` tell us that, if t_0 has a product type $T_1 * T_2$ (i.e., if it will reduce to a pair), then the types of the projections from this pair are T_1 and T_2 .

Unit

Another handy base type, found especially in functional languages, is the singleton type `Unit`. It has a single element -- the term constant `unit` (with a small *u*) -- and a typing rule making `unit` an element of `Unit`. We also add `unit` to the set of possible values -- indeed, `unit` is the *only* possible result of reducing an expression of type `Unit`.

Syntax:

$t ::=$	Terms
...	(other terms same as before)
<code>unit</code>	<code>unit</code>

$v ::=$	Values
...	
<code>unit</code>	unit value
$T ::=$	Types
...	
<code>Unit</code>	unit type

Typing:

$$\frac{}{\Gamma \vdash \text{unit} \in \text{Unit}} \quad (T_Unit)$$

It may seem a little strange to bother defining a type that has just one element -- after all, wouldn't every computation living in such a type be trivial?

This is a fair question, and indeed in the STLC the `Unit` type is not especially critical (though we'll see two uses for it below). Where `Unit` really comes in handy is in richer languages with *side effects* -- e.g., assignment statements that mutate variables or pointers, exceptions and other sorts of nonlocal control structures, etc. In such languages, it is convenient to have a type for the (trivial) result of an expression that is evaluated only for its effect.

+

No! For instance $(\lambda x:\text{Unit}.x)$ `unit` is also a *term* of type `unit`.

Sums

Many programs need to deal with values that can take two distinct forms. For example, we might identify students in a university database using *either* their name *or* their id number. A search function might return *either* a matching value *or* an error code.

These are specific examples of a binary *sum type* (sometimes called a *disjoint union*), which describes a set of values drawn from one of two given types, e.g.:

`Nat + Bool`

We create elements of these types by *tagging* elements of the component types. For example, if `n` is a `Nat` then `inl n` is an element of `Nat+Bool`; similarly, if `b` is a `Bool` then `inr b` is a `Nat+Bool`. The names of the tags `inl` and `inr` arise from thinking of them as functions

`inl` $\in \text{Nat} \rightarrow \text{Nat} + \text{Bool}$

`inr` $\in \text{Bool} \rightarrow \text{Nat} + \text{Bool}$

that "inject" elements of `Nat` or `Bool` into the left and right components of the sum type `Nat+Bool`. (But note that we don't actually treat them as functions in the way we formalize them: `inl` and `inr` are keywords, and `inl t` and `inr t` are primitive syntactic forms, not function applications.)

In general, the elements of a type $T_1 + T_2$ consist of the elements of T_1 tagged with the token `inl`, plus the elements of T_2 tagged with `inr`.

As we've seen in Rocq programming, one important use of sums is signaling errors:

`div` $\in \text{Nat} \rightarrow \text{Nat} \rightarrow (\text{Nat} + \text{Unit})$

`div` =

```
\x:Nat, \y:Nat,
  if iszero y then
    inr unit
  else
    inl ...
```

The type `Nat + Unit` above is in fact isomorphic to `option nat` in Rocq -- i.e., it's easy

to write functions that translate back and forth.

To *use* elements of sum types, we introduce a **case** construct (a very simplified form of Rocq's **match**) to destruct them. For example, the following procedure converts a **Nat+Bool** into a **Nat**:

```
getNat ∈ Nat+Bool -> Nat
getNat =
  \x:Nat+Bool,
    case x of
      inl n => n
    | inr b => if b then 1 else 0
```

More formally...

Syntax:

t ::=	Terms
...	(other terms same as before)
inl T ₂ t ₁	tagging (left)
inr T ₁ t ₂	tagging (right)
case t ₀ of	case analysis
inl x ₁ => t ₁	
inr x ₂ => t ₂	

v ::=	Values
...	
inl T ₂ v ₁	tagged value (left)
inr T ₁ v ₂	tagged value (right)

T ::=	Types
...	
T ₁ + T ₂	sum type

Reduction:

$$\frac{t_1 \rightarrow t_1'}{\text{inl } T_2 \ t_1 \rightarrow \text{inl } T_2 \ t_1'} \quad (\text{ST_Inl})$$

$$\frac{t_2 \rightarrow t_2'}{\text{inr } T_1 \ t_2 \rightarrow \text{inr } T_1 \ t_2'} \quad (\text{ST_Inr})$$

$$\frac{t_0 \rightarrow t_0'}{\text{case } t_0 \text{ of inl } x_1 \Rightarrow t_1 \mid \text{inr } x_2 \Rightarrow t_2 \rightarrow \text{case } t_0' \text{ of inl } x_1 \Rightarrow t_1 \mid \text{inr } x_2 \Rightarrow t_2} \quad (\text{ST_Case})$$

$$\frac{}{\text{case (inl } T_2 \ v_1) \text{ of inl } x_1 \Rightarrow t_1 \mid \text{inr } x_2 \Rightarrow t_2 \rightarrow [x_1 := v_1]t_1} \quad (\text{ST_CaseInl})$$

$$\frac{}{\text{case (inr } T_1 \ v_2) \text{ of inl } x_1 \Rightarrow t_1 \mid \text{inr } x_2 \Rightarrow t_2 \rightarrow [x_2 := v_2]t_2} \quad (\text{ST_CaseInr})$$

Typing:

$$\frac{}{\Gamma \vdash t_1 \in T_1} \quad (\text{T_Inl})$$

$$\begin{array}{c}
\text{Gamma} \vdash \text{inl } T_2 \ t_1 \in T_1 + T_2 \\
\\
\frac{\text{Gamma} \vdash t_2 \in T_2}{\text{Gamma} \vdash \text{inr } T_1 \ t_2 \in T_1 + T_2} \text{ (T_Inr)} \\
\\
\frac{\begin{array}{c} \text{Gamma} \vdash t_0 \in T_1 + T_2 \\ x_1 \mapsto T_1; \text{Gamma} \vdash t_1 \in T_3 \\ x_2 \mapsto T_2; \text{Gamma} \vdash t_2 \in T_3 \end{array}}{\text{Gamma} \vdash \text{case } t_0 \text{ of } \text{inl } x_1 \Rightarrow t_1 \mid \text{inr } x_2 \Rightarrow t_2 \in T_3} \text{ (T_Case)}
\end{array}$$

We use the type annotations on `inl` and `inr` to make the typing relation deterministic (each term has at most one type), as we did for functions.

Without this extra information, the typing rule `T_Inl`, for example, would have to say that, once we have shown that t_1 is an element of type T_1 , we can derive that `inl  $t_1$` is an element of $T_1 + T_2$ for *any* type T_2 . For example, we could derive both `inl 5 : Nat + Nat` and `inl 5 : Nat + Bool` (and infinitely many other types). This peculiarity (technically, a failure of uniqueness of types) would mean that we cannot build a typechecking algorithm simply by "reading the rules from bottom to top" as we could for all the other features seen so far.

There are various ways to deal with this difficulty. One simple one -- which we've adopted here -- forces the programmer to explicitly annotate the "other side" of a sum type when performing an injection. This is a bit heavy for programmers (so real languages adopt other solutions), but it is easy to understand and formalize.

Lists

The typing features we have seen can be classified into *base types* like `Bool`, and *type constructors* like `→` and `×` that build new types from old ones. Another useful type constructor is `List`. For every type T , the type `List T` describes finite-length lists whose elements are drawn from T .

In principle, we could encode lists using pairs, sums, unit, and *recursive* types. But giving semantics to recursive types is non-trivial. Instead, we'll just discuss the special case of lists directly.

Below we give the syntax, semantics, and typing rules for lists. Except for the fact that explicit type annotations are mandatory on `nil` and cannot appear on `cons`, these lists are essentially identical to those we built in Rocq. We use `case`, rather than `head` and `tail` operators, to destruct lists, to avoid dealing with questions like "what is the head of the empty list?"

For example, here is a function that calculates the sum of the first two elements of a list of numbers:

```

\ x : List Nat,
case x of
  nil    => 0
  | a :: x' => case x' of
    nil    => a
    | b :: x'' => a + b

```

Syntax:

<code>t ::=</code>	Terms
<code>...</code>	
<code>nil T</code>	empty list
<code>t₁ :: t₂</code>	cons

case t_1 of	case analysis
nil	$\Rightarrow t_2$
$ \text{ xh}::\text{xt}$	$\Rightarrow t_3$

$v ::=$	Values
$ \dots$	
$ \text{ nil } T$	nil value
$ v_1 :: v_2$	cons value

$T ::=$	Types
$ \dots$	
$ \text{ List } T$	list of T s

Reduction:

$$\frac{t_1 \rightarrow t_1'}{t_1 :: t_2 \rightarrow t_1' :: t_2} \quad (\text{ST_Cons1})$$

$$\frac{t_2 \rightarrow t_2'}{v_1 :: t_2 \rightarrow v_1 :: t_2'} \quad (\text{ST_Cons2})$$

$$\frac{t_1 \rightarrow t_1'}{(\text{case } t_1 \text{ of } \text{nil} \Rightarrow t_2 \mid \text{xh}::\text{xt} \Rightarrow t_3) \rightarrow (\text{case } t_1' \text{ of } \text{nil} \Rightarrow t_2 \mid \text{xh}::\text{xt} \Rightarrow t_3)} \quad (\text{ST_Lcase1})$$

$$\frac{}{(\text{case } \text{nil } T_1 \text{ of } \text{nil} \Rightarrow t_2 \mid \text{xh}::\text{xt} \Rightarrow t_3) \rightarrow t_2} \quad (\text{ST_LcaseNil})$$

$$\frac{}{(\text{case } (v_1::v_2) \text{ of } \text{nil} \Rightarrow t_2 \mid \text{xh}::\text{xt} \Rightarrow t_3) \rightarrow [\text{xh}:=v_1][\text{xt}:=v_2]t_3} \quad (\text{ST_LcaseCons})$$

Typing:

$$\frac{}{\text{Gamma} \vdash \text{nil } T_1 \in \text{List } T_1} \quad (\text{T_Nil})$$

$$\frac{\text{Gamma} \vdash t_1 \in T_1 \quad \text{Gamma} \vdash t_2 \in \text{List } T_1}{\text{Gamma} \vdash t_1 :: t_2 \in \text{List } T_1} \quad (\text{T_Cons})$$

$$\frac{\begin{array}{c} \text{Gamma} \vdash t_1 \in \text{List } T_1 \\ \text{Gamma} \vdash t_2 \in T_2 \\ (\text{xh} \mapsto T_1; \text{xt} \mapsto \text{List } T_1; \text{Gamma}) \vdash t_3 \in T_2 \end{array}}{\text{Gamma} \vdash (\text{case } t_1 \text{ of } \text{nil} \Rightarrow t_2 \mid \text{xh}::\text{xt} \Rightarrow t_3) \in T_2} \quad (\text{T_Lcase})$$

General Recursion

Another facility found in most programming languages (including Rocq) is the ability to define recursive functions. For example, we would like to be able to define and use the factorial function like this:

```
let fact = \x:Nat,
    if x=0 then 1 else x * (fact (pred x))) in
fact 3.
```

Note that the right-hand side of this binder mentions `fact`, the variable being bound -- something that is not allowed according to the way we defined `let` above.

(The body of a `let` is typechecked in the same context as the `let` itself, which means that the recursive occurrence of `fact` in the body will not have a type in the context when it is looked up by the `T_Var` rule.)

Changing the `let` rule to handle "recursive definitions" like this is possible, but it requires some extra effort -- e.g., passing around an extra "environment" of recursive function definitions in the definition of the `step` relation. We're going to take a simpler path here.

Here is another way of presenting recursive functions that is a bit more verbose but equally powerful and much more straightforward to formalize: instead of writing recursive definitions, we will define a *fixed-point operator* called `fix` that performs the "unfolding" of the recursive definition in the right-hand side as needed, during reduction.

For example, instead of

```
fact = \x:Nat,
      if x=0 then 1 else x * (fact (pred x))
```

we will write:

```
fact =
  fix
    (\f:Nat->Nat,
     \x:Nat,
     if x=0 then 1 else x * (f (pred x)))
```

We can derive the latter from the former as follows:

- In the right-hand side of the definition of `fact`, replace recursive references to `fact` by a fresh variable `f`.
- Add an abstraction binding `f` at the front, with an appropriate type annotation. (Since we are using `f` in place of `fact`, which had type `Nat→Nat`, we should require `f` to have the same type.) The new abstraction has type `(Nat→Nat) → (Nat→Nat)`.
- Apply `fix` to this abstraction. This application has type `Nat→Nat`.
- Use all of this as the right-hand side of an ordinary `let`-binding for `fact`.

For the mathematically inclined, the intuition here is that the higher-order function `f` passed to `fix` is a *generator* for the `fact` function: if `f` is applied to a function that "approximates" the desired behavior of `fact` up to some number `n` (that is, a function that returns correct results on inputs less than or equal to `n` but we don't care what it does on inputs greater than `n`), then `f` returns a slightly better approximation to `fact` -- a function that returns correct results for inputs up to `n+1`. Applying `fix` to this generator returns its *fixed point*, which is a function that gives the desired behavior for all inputs `n`.

(The term "fixed point" is used here in exactly the same sense as in ordinary mathematics, where a fixed point of a function `f` is an input `x` such that `f(x) = x`. Here, a fixed point of a function `F` of type `(Nat→Nat) → (Nat→Nat)` is a function `f` of type `Nat→Nat` such that `F f` behaves the same as `f`.)

Syntax:

<code>t ::=</code>	Terms
...	
<code>fix t₁</code>	fixed-point operator

Reduction:

$$\frac{t_1 \rightarrow t_1'}{\text{fix } t_1 \rightarrow \text{fix } t_1'} \quad (\text{ST_Fix1})$$

$$\frac{}{\text{fix } (\backslash x f : T_1. t_1) \rightarrow [x f := \text{fix } (\backslash x f : T_1. t_1)] t_1} \quad (\text{ST_FixAbs})$$

Typing:

$$\frac{\Gamma \mid \vdash t_1 \in T_1 \rightarrow T_1}{\Gamma \mid \vdash \text{fix } t_1 \in T_1} \quad (\text{T_Fix})$$

Let's see how ST_FixAbs works by reducing `fact 3 = fix F 3`, where

`F = (\f. \x. if x=0 then 1 else x * (f (pred x)))`

(type annotations are omitted for brevity).

```

fix F 3
--> ST_FixAbs + ST_App1
  (\x. if x=0 then 1 else x * (fix F (pred x))) 3
--> ST_AppAbs
  if 3=0 then 1 else 3 * (fix F (pred 3))
--> ST_If0_Nonzero
  3 * (fix F (pred 3))
--> ST_FixAbs + ST_Mult2 + ST_App1
  3 * ((\x. if x=0 then 1 else x * (fix F (pred x))) (pred 3))
--> ST_PredNat + ST_Mult2 + ST_App2
  3 * ((\x. if x=0 then 1 else x * (fix F (pred x))) 2)
--> ST_AppAbs + ST_Mult2
  3 * (if 2=0 then 1 else 2 * (fix F (pred 2)))
--> ST_If0_Nonzero + ST_Mult2
  3 * (2 * (fix F (pred 2)))
--> ST_FixAbs + 2 x ST_Mult2 + ST_App1
  3 * (2 * ((\x. if x=0 then 1 else x * (fix F (pred x))) (pred 2)))
--> ST_PredNat + 2 x ST_Mult2 + ST_App2
  3 * (2 * ((\x. if x=0 then 1 else x * (fix F (pred x))) 1))
--> ST_AppAbs + 2 x ST_Mult2
  3 * (2 * (if 1=0 then 1 else 1 * (fix F (pred 1))))
--> ST_If0_Nonzero + 2 x ST_Mult2
  3 * (2 * (1 * (fix F (pred 1))))
--> ST_FixAbs + 3 x ST_Mult2 + ST_App1
  3 * (2 * (1 * ((\x. if x=0 then 1 else x * (fix F (pred x))) (pred 1))))
--> ST_PredNat + 3 x ST_Mult2 + ST_App2
  3 * (2 * (1 * ((\x. if x=0 then 1 else x * (fix F (pred x))) 0)))
--> ST_AppAbs + 3 x ST_Mult2
  3 * (2 * (1 * (if 0=0 then 1 else 0 * (fix F (pred 0)))))
--> ST_If0Zero + 3 x ST_Mult2
  3 * (2 * (1 * 1))
--> ST_MultNats + 2 x ST_Mult2
  3 * (2 * 1)
--> ST_MultNats + ST_Mult2
  3 * 2
--> ST_MultNats
  6

```

The simply typed lambda-calculus with fixed points is a famous and extensively studied

system. It is often called *PCF* because it is a simple language of "partial computable functions".

Exercise: 1 star, standard, optional (halve_fix)

Translate this informal recursive definition into one using `fix`:

```
halve =
  \x:Nat,
    if x=0 then 0
    else if (pred x)=0 then 0
    else 1 + (halve (pred (pred x)))
(* FILL IN HERE *)
```

□

Exercise: 1 star, standard, optional (fact_steps)

Write down the sequence of steps that the term `fact 1` goes through to reduce to a normal form (assuming the usual reduction rules for arithmetic operations).

(* FILL IN HERE *)

□

The ability to form the fixed point of a function of type $T \rightarrow T$ for any T has some surprising consequences. In particular, it implies that *every* type is inhabited by some term. To see this, observe that, for every type T , we can define the term

```
fix (\x:T, x)
```

By T_Fix and T_Abs , this term has type T . By ST_FixAbs it reduces to itself, over and over again. Thus it is a *diverging element* of T .

More usefully, here's an example using `fix` to define a two-argument recursive function:

```
equal =
  fix
    (\eq:Nat->Nat->Bool,
      \m:Nat, \n:Nat,
        if m=0 then iszero n
        else if n=0 then false
        else eq (pred m) (pred n))
```

And finally, here is an example where `fix` is used to define a *pair* of recursive functions (illustrating the fact that the type T_1 in the rule T_Fix need not be a function type):

```
let evenodd =
  fix
    (\eo: ((Nat -> Nat) * (Nat -> Nat)),
      (\n:Nat, if_0 n then 1 else (eo.snd (pred n)),
        \n:Nat, if_0 n then 0 else (eo.fst (pred n)))) in
let even = evenodd.fst in
let odd  = evenodd.snd in
(even 3, even 4)}
```

Records

As a final example of a basic extension of the STLC, let's look briefly at how to define *records* and their types. Intuitively, records can be obtained from pairs by two straightforward generalizations: they are n -ary (rather than just binary) and their fields are accessed by *label* (rather than position).

Syntax:

```
t ::=
  | ...
```

Terms

{i ₁ =t ₁ , ..., in=tn}	record
t ₀ .i	projection
v ::=	Values
...	
{i ₁ =v ₁ , ..., in=vn}	record value
T ::=	Types
...	
{i ₁ :T ₁ , ..., in:Tn}	record type

The generalization from products should be pretty obvious. But it's worth noticing the ways in which what we've actually written is even *more* informal than the informal syntax we've used in previous sections and chapters: we've used "..." in several places to mean "any number of these," and we've omitted explicit mention of the usual side condition that the labels of a record should not contain any repetitions.

Reduction:

$$\begin{array}{c}
 \frac{ti \rightarrow ti'}{\{i_1=v_1, \dots, im=vm, in=ti, \dots\} \rightarrow \{i_1=v_1, \dots, im=vm, in=ti', \dots\}} \quad (ST_Rcd) \\
 \\
 \frac{t_0 \rightarrow t_0'}{t_0.i \rightarrow t_0'.i} \quad (ST_Proj1) \\
 \\
 \frac{}{\{..., i=vi, \dots\}.i \rightarrow vi} \quad (ST_ProjRcd)
 \end{array}$$

Again, these rules are a bit informal. For example, the first rule is intended to be read "if *ti* is the leftmost field that is not a value and if *ti* steps to *ti'*, then the whole record steps..." In the last rule, the intention is that there should be only one field called *i*, and that all the other fields must contain values.

The typing rules are also simple:

$$\begin{array}{c}
 \frac{\Gamma \vdash t_1 \in T_1 \quad \dots \quad \Gamma \vdash t_n \in T_n}{\Gamma \vdash \{i_1=t_1, \dots, in=tn\} \in \{i_1:T_1, \dots, in:Tn\}} \quad (T_Rcd) \\
 \\
 \frac{\Gamma \vdash t_0 \in \{..., i:Ti, \dots\}}{\Gamma \vdash t_0.i \in Ti} \quad (T_Proj)
 \end{array}$$

There are several ways to approach formalizing the above definitions.

- We can directly formalize the syntactic forms and inference rules, staying as close as possible to the form we've given them above. This is conceptually straightforward, and it's probably what we'd want to do if we were building a real compiler (in particular, it will allow us to print error messages in the form that programmers will find easy to understand). But the formal versions of the rules will not be very pretty or easy to work with, because all the ...s above will have to be replaced with explicit quantifications or comprehensions. For this reason, records are not included in the extended exercise at the end of this chapter. (It is still useful to discuss them informally here because they will help motivate the addition of subtyping to the type system when we get to the [Sub](#) chapter.)
- Alternatively, we could look for a smoother way of presenting records -- for

example, a binary presentation with one constructor for the empty record and another constructor for adding a single field to an existing record, instead of a single monolithic constructor that builds a whole record at once. This is the right way to go if we are primarily interested in studying the metatheory of the calculi with records, since it leads to clean and elegant definitions and proofs. Chapter [Records](#) shows how this can be done.

- Finally, if we like, we can avoid formalizing records altogether, by stipulating that record notations are just informal shorthands for more complex expressions involving pairs and product types. We sketch this approach in the next section.

Encoding Records (Optional)

Let's see how records can be encoded using just pairs and `unit`. (This clever encoding, as well as the observation that it also extends to systems with subtyping, is due to Luca Cardelli.)

First, observe that we can encode arbitrary-size *tuples* using nested pairs and the `unit` value. To avoid overloading the pair notation (t_1, t_2) , we'll use curly braces without labels to write down tuples, so $\{\}$ is the empty tuple, $\{5\}$ is a singleton tuple, $\{5, 6\}$ is a 2-tuple (morally the same as a pair), $\{5, 6, 7\}$ is a triple, etc.

```
{}          ---> unit
{t1, t2, ..., tn} ---> (t1, trest)
                                where {t2, ..., tn} ---> trest
```

Similarly, we can encode tuple types using nested product types:

```
{ }          ---> Unit
{T1, T2, ..., Tn} ---> T1 * TRest
                                where {T2, ..., Tn} ---> TRest
```

The operation of projecting a field from a tuple can be encoded using a sequence of second projections followed by a first projection:

```
t.0          ---> t.fst
t.(n+1)      ---> (t.snd).n
```

Next, suppose that there is some total ordering on record labels, so that we can associate each label with a unique natural number. This number is called the *position* of the label. For example, we might assign positions like this:

LABEL	POSITION
a	0
b	1
c	2
...	...
bar	1395
...	...
foo	4460
...	...

We use these positions to encode record values as tuples (i.e., as nested pairs) by sorting the fields according to their positions. For example:

```
{a=5,b=6}      ---> {5,6}
{a=5,c=7}      ---> {5,unit,7}
{c=7,a=5}      ---> {5,unit,7}
{c=5,b=3}      ---> {unit,3,5}
{f=8,c=5,a=7}  ---> {7,unit,5,unit,unit,8}
{f=8,c=5}      ---> {unit,unit,5,unit,unit,8}
```

Note that each field appears in the position associated with its label, that the size of the tuple is determined by the label with the highest position, and that we fill in unused positions with `unit`.

We do exactly the same thing with record types:

$\{a:\text{Nat}, b:\text{Nat}\}$	$\dashrightarrow$	$\{\text{Nat}, \text{Nat}\}$
$\{c:\text{Nat}, a:\text{Nat}\}$	$\dashrightarrow$	$\{\text{Nat}, \text{Unit}, \text{Nat}\}$
$\{f:\text{Nat}, c:\text{Nat}\}$	$\dashrightarrow$	$\{\text{Unit}, \text{Unit}, \text{Nat}, \text{Unit}, \text{Unit}, \text{Nat}\}$

Finally, record projection is encoded as a tuple projection from the appropriate position:

$$t.l \dashrightarrow t.(\text{position of } l)$$

It is not hard to check that all the typing rules for the original "direct" presentation of records are validated by this encoding. (The reduction rules are "almost validated" -- not quite, because the encoding reorders fields.)

Of course, this encoding will not be very efficient if we happen to use a record with label `foo`! But things are not actually as bad as they might seem: for example, if we assume that our compiler can see the whole program at the same time, we can *choose* the numbering of labels so that we assign small positions to the most frequently used labels. Indeed, there are industrial compilers that essentially do this!

Variants (Optional)

Just as products can be generalized to records, sums can be generalized to n-ary labeled types called *variants*. Instead of $T_1 + T_2$, we can write something like

$\langle l_1:T_1, l_2:T_2, \dots, l_n:T_n \rangle$ where $l_1, l_2, \dots$ are field labels which are used both to build instances and as case arm labels.

These n-ary variants give us almost enough mechanism to build arbitrary inductive data types like lists and trees from scratch -- the only thing missing is a way to allow *recursion* in type definitions. We won't cover this here, but detailed treatments can be found in many textbooks -- e.g., *Types and Programming Languages* [Pierce 2002].

Exercise: Formalizing the Extensions

Module `STLCExtended`.

In this series of exercises, you will formalize some of the extensions described in this chapter. We've provided the necessary additions to the syntax of terms and types, and we've included a few examples that you can test your definitions with to make sure they are working as expected. You'll fill in the rest of the definitions and extend all the proofs accordingly.

To get you started, we've provided implementations for:

- numbers
- sums
- lists
- unit

You need to complete the implementations for:

- pairs
- let (which involves binding)
- fix

A good strategy is to work on the extensions one at a time (first pairs, then let, then fix), in separate passes, rather than trying to do all three at once in a single pass. For each definition or proof, begin by reading carefully through the parts that are provided for you, referring to the text in the `Stlc` chapter for high-level intuitions and the embedded comments for detailed mechanics.

Syntax

Inductive `ty : Type :=`

```

| Ty_Arrow : ty → ty → ty
| Ty_Nat : ty
| Ty_Sum : ty → ty → ty
| Ty_List : ty → ty
| Ty_Unit : ty
| Ty_Prod : ty → ty → ty.

Inductive tm : Type :=
(* pure STLC *)
| tm_var : string → tm
| tm_app : tm → tm → tm
| tm_abs : string → ty → tm → tm
(* numbers *)
| tm_const : nat → tm
| tm_succ : tm → tm
| tm_pred : tm → tm
| tm_mult : tm → tm → tm
| tm_if0 : tm → tm → tm → tm
(* sums *)
| tm_inl : ty → tm → tm
| tm_inr : ty → tm → tm
| tm_case : tm → string → tm → string → tm → tm
(* i.e., case t0 of inl x1 ⇒ t1 | inr x2 ⇒ t2 *)
(* lists *)
| tm_nil : ty → tm
| tm_cons : tm → tm → tm
| tm_lcase : tm → tm → string → string → tm → tm
(* i.e., case t1 of | nil ⇒ t2 | x::y ⇒ t3 *)
(* unit *)
| tm_unit : tm

(* You are going to be working on the following extensions: *)

(* pairs *)
| tm_pair : tm → tm → tm
| tm_fst : tm → tm
| tm_snd : tm → tm
(* let *)
| tm_let : string → tm → tm → tm
(* i.e., let x = t1 in t2 *)
(* fix *)
| tm_fix : tm → tm.

```

Note that, for brevity, we've omitted booleans and instead provided a single `if0` form combining a zero test and a conditional. That is, instead of writing

`if x = 0 then ... else ...`

we'll write this:

`if0 x then ... else ...`

```

Definition w : string := "w".
Definition x : string := "x".
Definition y : string := "y".
Definition z : string := "z".

Hint Unfold x : core.
Hint Unfold y : core.
Hint Unfold z : core.

Delimit Scope stlc_scope with stlc.
Open Scope stlc_scope.

Notation "x" := x (in custom stlc_ty at level 0, x global) : stlc_scope.

Notation "<[ x ]>" := x (x custom stlc_ty).

Notation "( t )" := t (in custom stlc_ty at level 0, t custom stlc_ty) :
stlc_scope.
Notation "S → T" := (Ty_Arrow S T) (in custom stlc_ty at level 99, right
associativity) : stlc_scope.

Notation "$ ( t )" := t (in custom stlc_ty at level 0, t constr) : stlc_scope.

```

```

Notation "x" := x (in custom stlc_tm at level 0, x global) : stlc_scope.
Notation "'_" := _ (in custom stlc_tm at level 0) : stlc_scope.
Notation "'$' x" := x (in custom stlc_tm at level 0, x constr at level 0) :
stlc_scope.
Notation "'$( ' x ' )'" := x (in custom stlc_tm at level 0, x constr, only
parsing) : stlc_scope.
Notation "<{ e }>" := e (e custom stlc_tm at level 200) : stlc_scope.
Notation "( x )" := x (in custom stlc_tm at level 0, x custom stlc_tm) :
stlc_scope.

Notation "x y" := (tm_app x y) (in custom stlc_tm at level 10, left
associativity) : stlc_scope.
Notation "\ x : t , y" :=
  (tm_abs x t y) (in custom stlc_tm at level 200, x global,
                  t custom stlc_ty,
                  y custom stlc_tm at level 200,
                  left associativity).

Coercion tm_var : string >> tm.
Arguments tm_var _%string.

Notation "'Nat'" := Ty_Nat (in custom stlc_ty at level 0).
Notation "'succ' x" := (tm_succ x) (in custom stlc_tm at level 10,
                                   x custom stlc_tm at level 0) : stlc_scope.
Notation "'pred' x" := (tm_pred x) (in custom stlc_tm at level 10,
                                   x custom stlc_tm at level 0) : stlc_scope.
Notation "x * y" := (tm_mult x y) (in custom stlc_tm at level 95,
                                   right associativity) : stlc_scope.
Notation "'if0' x 'then' y 'else' z" :=
  (tm_if0 x y z) (in custom stlc_tm at level 0,
                  x custom stlc_tm at level 0,
                  y custom stlc_tm at level 0,
                  z custom stlc_tm at level 0) : stlc_scope.

Coercion tm_const : nat >> tm.

Notation "S + T" :=
  (Ty_Sum S T) (in custom stlc_ty at level 3, left associativity).
Notation "'inl' T t" := (tm_inl T t) (in custom stlc_tm at level 10,
                                     T custom stlc_ty,
                                     t custom stlc_tm at level 0).
Notation "'inr' T t" := (tm_inr T t) (in custom stlc_tm at level 10,
                                     T custom stlc_ty,
                                     t custom stlc_tm at level 0).
Notation "'case' t0 'of' '|' 'inl' x1 '=>' t1 '|' 'inr' x2 '=>' t2" :=
  (tm_case t0 x1 t1 x2 t2) (in custom stlc_tm at level 200,
                              t0 custom stlc_tm at level 200,
                              x1 global,
                              t1 custom stlc_tm at level 200,
                              x2 global,
                              t2 custom stlc_tm at level 200,
                              left associativity).

Notation "X * Y" :=
  (Ty_Prod X Y) (in custom stlc_ty at level 2, X custom stlc_ty, Y custom
stlc_ty at level 0) : stlc_scope.

Notation "( x ' , ' y )" := (tm_pair x y) (in custom stlc_tm at level 0,
                                           x custom stlc_tm,
                                           y custom stlc_tm) : stlc_scope.

Notation "t '.fst'" := (tm_fst t) (in custom stlc_tm at level 1) : stlc_scope.
Notation "t '.snd'" := (tm_snd t) (in custom stlc_tm at level 1) : stlc_scope.

Notation "'List' T" :=
  (Ty_List T) (in custom stlc_ty at level 4) : stlc_scope.
Notation "'List' T" := (Ty_List T) (at level 0) : stlc_scope.
Notation "'nil' T" := (tm_nil T) (in custom stlc_tm at level 0, T custom
stlc_ty) : stlc_scope.
Notation "h '::' t" := (tm_cons h t) (in custom stlc_tm at level 2, right
associativity) : stlc_scope.
Notation "'case' t1 'of' '|' 'nil' '=>' t2 '|' x '::' y '=>' t3" :=

```

```

(tm_lcase t1 t2 x y t3) (in custom stlc_tm at level 200,
    t1 custom stlc_tm at level 200,
    t2 custom stlc_tm at level 0,
    x global,
    y global,
    t3 custom stlc_tm at level 0,
    left associativity) : stlc_scope.

Notation "'Unit'" :=
  (Ty_Unit) (in custom stlc_ty at level 0) : stlc_scope.
Notation "'unit'" := tm_unit (in custom stlc_tm at level 0) : stlc_scope.

Notation "'let' x '=' t1 'in' t2" :=
  (tm_let x t1 t2) (in custom stlc_tm at level 200) : stlc_scope.

Notation "'fix' t" := (tm_fix t) (in custom stlc_tm at level 200) : stlc_scope.

```

Substitution

```

Reserved Notation "'[' x ':' s ']' t" (in custom stlc_tm at level 5, x global,
  s custom stlc_tm,
  t custom stlc_tm at next level, right associativity).

```

Exercise: 3 stars, standard (STLCExtended.subst)

```

Fixpoint subst (x : string) (s : tm) (t : tm) : tm :=
  match t with
  (* pure STLC *)
  | tm_var y =>
    if String.eqb x y then s else t
  | <{\y:T, t1}> =>
    if String.eqb x y then t else <{\y:T, [x:=s] t1}>
  | <{t1 t2}> =>
    <{([x:=s] t1) ([x:=s] t2)}>
  (* numbers *)
  | tm_const _ =>
    t
  | <{succ t1}> =>
    <{succ ([x := s] t1)}>
  | <{pred t1}> =>
    <{pred ([x := s] t1)}>
  | <{t1 × t2}> =>
    <{ ([x := s] t1) × ([x := s] t2)}>
  | <{if0 t1 then t2 else t3}> =>
    <{if0 [x := s] t1 then [x := s] t2 else [x := s] t3}>
  (* sums *)
  | <{inl T2 t1}> =>
    <{inl T2 ( [x:=s] t1)}>
  | <{inr T1 t2}> =>
    <{inr T1 ( [x:=s] t2)}>
  | <{case t0 of | inl y1 => t1 | inr y2 => t2}> =>
    <{case ([x:=s] t0) of
      | inl y1 => $(if String.eqb x y1 then t1 else <{ [x:=s] t1 }> )
      | inr y2 => $(if String.eqb x y2 then t2 else <{ [x:=s] t2 }> )}>
  (* lists *)
  | <{nil T}> =>
    t
  | <{t1 :: t2}> =>
    <{ ([x:=s] t1) :: [x:=s] t2}>
  | <{case t1 of | nil => t2 | y1 :: y2 => t3}> =>
    <{case ( [x:=s] t1 ) of
      | nil => [x:=s] t2
      | y1 :: y2 =>
        $(if String.eqb x y1 then
          t3

```

```

        else if String.eqb x y2 then t3
        else <{ [x:=s] t3 }> }>

(* unit *)
| <{unit}> ⇒ <{unit}>

(* Complete the following cases. *)

(* pairs *)
(* FILL IN HERE *)
(* let *)
(* FILL IN HERE *)
(* fix *)
(* FILL IN HERE *)
| _ ⇒ t (* ... and delete this line when you finish the exercise *)
end

where "'[' x ':' s ']' t" := (subst x s t) (in custom stlc_tm) : stlc_scope.

(* Make sure the following tests are valid by reflexivity: *)
Example substeg1 :
  <{ [z:=$0] (let w = z in z) }> = <{ let w = $0 in $0 }>.
Proof.
(*
  reflexivity.
*)
(* FILL IN HERE *) Admitted.

Example substeg2 :
  <{ [z:=$0] (let w = z in w) }> = <{ let w = $0 in w }>.
Proof.
(*
  reflexivity.
*)
(* FILL IN HERE *) Admitted.

Example substeg3 :
  <{ [z:=$0] (let y = succ $0 in z) }> = <{ let y = succ $0 in $0 }>.
Proof.
(*
  reflexivity.
*)
(* FILL IN HERE *) Admitted.
□

```

Reduction

Next we define the values of our language.

```

Inductive value : tm → Prop :=
  (* In pure STLC, function abstractions are values: *)
  | v_abs : ∀ x T2 t1,
    value <{\x:T2, t1}>
  (* Numbers are values: *)
  | v_nat : ∀ n : nat,
    value <{n}>
  (* A tagged value is a value: *)
  | v_inl : ∀ v T1,
    value v →
    value <{inl T1 v}>
  | v_inr : ∀ v T1,
    value v →
    value <{inr T1 v}>
  (* A list is a value iff its head and tail are values: *)
  | v_lnil : ∀ T1, value <{nil T1}>
  | v_lcons : ∀ v1 v2,
    value v1 →
    value v2 →
    value <{v1 :: v2}>
  (* A unit is always a value *)

```

```

| v_unit : value <{unit}>
(* A pair is a value if both components are: *)
| v_pair : ∀ v1 v2,
  value v1 →
  value v2 →
  value <{(v1, v2)}>.

```

Hint Constructors value : core.

Exercise: 3 stars, standard (STLCExtended.step)

```

Inductive step : tm → tm → Prop :=
(* pure STLC *)
| ST_AppAbs : ∀ x T2 t1 v2,
  value v2 →
  <{(\x:T2, t1) v2}> → <{ [x:=v2]t1 }>
| ST_App1 : ∀ t1 t1' t2,
  t1 → t1' →
  <{t1 t2}> → <{t1' t2}>
| ST_App2 : ∀ v1 t2 t2',
  value v1 →
  t2 → t2' →
  <{v1 t2}> → <{v1 t2'}>
(* numbers *)
| ST_Succ : ∀ t1 t1',
  t1 → t1' →
  <{succ t1}> → <{succ t1'}>
| ST_SuccNat : ∀ n : nat,
  <{succ n}> → <{ $(S n) }>
| ST_Pred : ∀ t1 t1',
  t1 → t1' →
  <{pred t1}> → <{pred t1'}>
| ST_PredNat : ∀ n:nat,
  <{pred n}> → <{ $(n - 1) }>
| ST_Mulconsts : ∀ n1 n2 : nat,
  <{n1 × n2}> → <{ $(n1 × n2) }>
| ST_Mult1 : ∀ t1 t1' t2,
  t1 → t1' →
  <{t1 × t2}> → <{t1' × t2}>
| ST_Mult2 : ∀ v1 t2 t2',
  value v1 →
  t2 → t2' →
  <{v1 × t2}> → <{v1 × t2'}>
| ST_If0 : ∀ t1 t1' t2 t3,
  t1 → t1' →
  <{if0 t1 then t2 else t3}> → <{if0 t1' then t2 else t3}>
| ST_If0_Zero : ∀ t2 t3,
  <{if0 $0 then t2 else t3}> → t2
| ST_If0_Nonzero : ∀ n t2 t3,
  <{if0 $(S n) then t2 else t3}> → t3
(* sums *)
| ST_Inl : ∀ t1 t1' T2,
  t1 → t1' →
  <{inl T2 t1}> → <{inl T2 t1'}>
| ST_Inr : ∀ t2 t2' T1,
  t2 → t2' →
  <{inr T1 t2}> → <{inr T1 t2'}>
| ST_Case : ∀ t0 t0' x1 t1 x2 t2,
  t0 → t0' →
  <{case t0 of | inl x1 ⇒ t1 | inr x2 ⇒ t2}> →
  <{case t0' of | inl x1 ⇒ t1 | inr x2 ⇒ t2}>

```

```

| ST_CaseInl : ∀ v₀ x₁ t₁ x₂ t₂ T₂,
  value v₀ →
  <{case inl T₂ v₀ of | inl x₁ ⇒ t₁ | inr x₂ ⇒ t₂}> → <{ [x₁:=v₀]t₁ }>
| ST_CaseInr : ∀ v₀ x₁ t₁ x₂ t₂ T₁,
  value v₀ →
  <{case inr T₁ v₀ of | inl x₁ ⇒ t₁ | inr x₂ ⇒ t₂}> → <{ [x₂:=v₀]t₂ }>
(* lists *)
| ST_Cons1 : ∀ t₁ t₁' t₂,
  t₁ → t₁' →
  <{t₁ :: t₂}> → <{t₁' :: t₂}>
| ST_Cons2 : ∀ v₁ t₂ t₂',
  value v₁ →
  t₂ → t₂' →
  <{v₁ :: t₂}> → <{v₁ :: t₂'}>
| ST_Lcase1 : ∀ t₁ t₁' t₂ x₁ x₂ t₃,
  t₁ → t₁' →
  <{case t₁ of | nil ⇒ t₂ | x₁ :: x₂ ⇒ t₃}> →
  <{case t₁' of | nil ⇒ t₂ | x₁ :: x₂ ⇒ t₃}>
| ST_LcaseNil : ∀ T₁ t₂ x₁ x₂ t₃,
  <{case nil T₁ of | nil ⇒ t₂ | x₁ :: x₂ ⇒ t₃}> → t₂
| ST_LcaseCons : ∀ v₁ vl t₂ x₁ x₂ t₃,
  value v₁ →
  value vl →
  <{case v₁ :: vl of | nil ⇒ t₂ | x₁ :: x₂ ⇒ t₃}>
  → <{ [x₂ := vl] ([x₁ := v₁] t₃) }>

(* Add rules for the following extensions. *)

(* pairs *)
(* FILL IN HERE *)
(* let *)
(* FILL IN HERE *)
(* fix *)
(* FILL IN HERE *)

where "t '→' t'" := (step t t').
□

Notation multistep := (multi step).
Notation "t₁ '→*' t₂" := (multistep t₁ t₂) (at level 40).

Hint Constructors step : core.

```

Typing

Definition context := partial_map ty.

Next we define the typing rules. These are nearly direct transcriptions of the inference rules shown above.

Exercise: 3 stars, standard (STLCExtended.has_type)

```

Inductive has_type : context → tm → ty → Prop :=
(* pure STLC *)
| T_Var : ∀ Gamma x T₁,
  Gamma x = Some T₁ →
  <{ Gamma |-- x \in T₁ }>
| T_Abs : ∀ Gamma x T₁ T₂ t₁,
  <{ x ↦ T₂ ; Gamma |-- t₁ \in T₁ }> →
  <{ Gamma |-- \x:T₂, t₁ \in T₂ → T₁ }>
| T_App : ∀ T₁ T₂ Gamma t₁ t₂,
  <{ Gamma |-- t₁ \in T₂ → T₁ }> →
  <{ Gamma |-- t₂ \in T₂ }> →
  <{ Gamma |-- t₁ t₂ \in T₁ }>

```

```

(* numbers *)
| T_Nat : ∀ Gamma (n : nat),
  <{ Gamma |-- n \in Nat }>
| T_Succ : ∀ Gamma t,
  <{ Gamma |-- t \in Nat }> →
  <{ Gamma |-- succ t \in Nat }>
| T_Pred : ∀ Gamma t,
  <{ Gamma |-- t \in Nat }> →
  <{ Gamma |-- pred t \in Nat }>
| T_Mult : ∀ Gamma t1 t2,
  <{ Gamma |-- t1 \in Nat }> →
  <{ Gamma |-- t2 \in Nat }> →
  <{ Gamma |-- t1 × t2 \in Nat }>
| T_If0 : ∀ Gamma t1 t2 t3 T0,
  <{ Gamma |-- t1 \in Nat }> →
  <{ Gamma |-- t2 \in T0 }> →
  <{ Gamma |-- t3 \in T0 }> →
  <{ Gamma |-- if0 t1 then t2 else t3 \in T0 }>

(* sums *)
| T_Inl : ∀ Gamma t1 T1 T2,
  <{ Gamma |-- t1 \in T1 }> →
  <{ Gamma |-- (inl T2 t1) \in T1 + T2 }>
| T_Inr : ∀ Gamma t2 T1 T2,
  <{ Gamma |-- t2 \in T2 }> →
  <{ Gamma |-- (inr T1 t2) \in T1 + T2 }>
| T_Case : ∀ Gamma t0 x1 T1 t1 x2 T2 t2 T3,
  <{ Gamma |-- t0 \in T1 + T2 }> →
  <{ x1 ⊢ T1 ; Gamma |-- t1 \in T3 }> →
  <{ x2 ⊢ T2 ; Gamma |-- t2 \in T3 }> →
  <{ Gamma |-- case t0 of | inl x1 ⇒ t1 | inr x2 ⇒ t2 \in T3 }>

(* lists *)
| T_Nil : ∀ Gamma T1,
  <{ Gamma |-- nil T1 \in List T1 }>
| T_Cons : ∀ Gamma t1 t2 T1,
  <{ Gamma |-- t1 \in T1 }> →
  <{ Gamma |-- t2 \in List T1 }> →
  <{ Gamma |-- t1 :: t2 \in List T1 }>
| T_Lcase : ∀ Gamma t1 T1 t2 x1 x2 t3 T2,
  <{ Gamma |-- t1 \in List T1 }> →
  <{ Gamma |-- t2 \in T2 }> →
  <{ x1 ⊢ T1 ; x2 ⊢ List T1 ; Gamma |-- t3 \in T2 }> →
  <{ Gamma |-- case t1 of | nil ⇒ t2 | x1 :: x2 ⇒ t3 \in T2 }>

(* unit *)
| T_Unit : ∀ Gamma,
  <{ Gamma |-- unit \in Unit }>

(* Add rules for the following extensions. *)

(* pairs *)
(* FILL IN HERE *)
(* let *)
(* FILL IN HERE *)
(* fix *)
(* FILL IN HERE *)

```

where "<{ Gamma ' |-- ' t ' ∈ ' T }>" := (has_type Gamma t T).

□

Hint Constructors has_type : core.

Examples

Exercise: 5 stars, standard, optional (STLCExtended examples)

This section presents formalized versions of the examples from above (plus several

more).

For each example, uncomment proofs and replace `Admitted` by `Qed` once you've implemented enough of the definitions for the tests to pass.

The examples at the beginning focus on specific features; you can use these to make sure your definition of a given feature is reasonable before moving on to extending the proofs later in the file with the cases relating to this feature. The later examples require all the features together, so you'll need to come back to these when you've got all the definitions filled in.

```
Module Examples.
```

Preliminaries

First, let's define a few variable names:

```
Open Scope string_scope.
Notation x := "x".
Notation y := "y".
Notation a := "a".
Notation f := "f".
Notation g := "g".
Notation l := "l".
Notation k := "k".
Notation i1 := "i1".
Notation i2 := "i2".
Notation processSum := "processSum".
Notation n := "n".
Notation eq := "eq".
Notation m := "m".
Notation evenodd := "evenodd".
Notation even := "even".
Notation odd := "odd".
Notation eo := "eo".
```

Next, a bit of Rocq hackery to automate searching for typing derivations. You don't need to understand this bit in detail -- just have a look over it so that you'll know what to look for if you ever find yourself needing to make custom extensions to `auto`.

The following `Hint` declarations say that, whenever `auto` arrives at a goal of the form $(\Gamma \vdash (tm_app\ e_1\ e_2) \in T)$, it should consider `eapply T_App`, leaving an existential variable for the middle type T_1 , and similar for `lcase`. That variable will then be filled in during the search for type derivations for e_1 and e_2 . We also include a hint to "try harder" when solving equality goals; this is useful to automate uses of `T_Var` (which includes an equality as a precondition).

```
Hint Extern 2 (has_type _ (tm_app _ _) _) =>
  eapply T_App; auto : core.
Hint Extern 2 (has_type _ (tm_lcase _ _ _ _) _) =>
  eapply T_Lcase; auto : core.
Hint Extern 2 (_ = _) => compute; reflexivity : core.
```

Numbers

```
Definition tm_test :=
  <{if_
    (pred
      (succ
        (pred
          ($2 × $0))))
    then $5
    else $6}>.

Example typechecks :
  <{ empty |-- tm_test \in Nat }>.
```

```

Proof.
  unfold tm_test.
  (* This typing derivation is quite deep, so we need
     to increase the max search depth of auto from the
     default 5 to 10. *)
  auto 10.
  (* FILL IN HERE *) Admitted.

Example reduces :
  tm_test ->* 5.
Proof.
  (*
    unfold tm_test. normalize.
  *)
  (* FILL IN HERE *) Admitted.

End Numtest.

```

Products

```

Module ProdTest.

Definition tm_test :=
  <{ (($5,$6),$7).fst.snd }>.

Example typechecks :
  <{ empty |-- tm_test \in Nat }>.
Proof. unfold tm_test. eauto. (* FILL IN HERE *) Admitted.

Example reduces :
  tm_test ->* 6.
Proof.
  (*
    unfold tm_test. normalize.
  *)
  (* FILL IN HERE *) Admitted.

End ProdTest.

```

let

```

Module LetTest.

Definition tm_test :=
  <{ let x = (pred $6) in
    (succ x) }>.

Example typechecks :
  <{ empty |-- tm_test \in Nat }>.
Proof. unfold tm_test. eauto.
  (* FILL IN HERE *) Admitted.

Example reduces :
  tm_test ->* 6.
Proof.
  (*
    unfold tm_test. normalize.
  *)
  (* FILL IN HERE *) Admitted.

End LetTest.

Module LetTest1.

Definition tm_test :=
  <{ let z = pred $6 in
    (succ z) }>.

Example typechecks :
  <{ empty |-- tm_test \in Nat }>.
Proof. unfold tm_test. eauto.
  (* FILL IN HERE *) Admitted.

```

```

Example reduces :
  tm_test ->* 6.
Proof.
(*
  unfold tm_test. normalize.
*)
(* FILL IN HERE *) Admitted.

End LetTest1.

```

Sums

```

Module Sumtest1.

Definition tm_test :=
  <{ case (inl Nat $5) of
    | inl x => x
    | inr y => y }>.

Example typechecks :
  <{ empty |-- tm_test \in Nat }>.
Proof. unfold tm_test. eauto. (* FILL IN HERE *) Admitted.

Example reduces :
  tm_test ->* 5.
Proof.
(*
  unfold tm_test. normalize.
*)
(* FILL IN HERE *) Admitted.

End Sumtest1.

Module Sumtest2.

(* let processSum =
  \x:Nat+Nat.
  case x of
  inl n => n
  inr n => tm_test0 n then 1 else 0 in
  (processSum (inl Nat 5), processSum (inr Nat 5)) *)

Definition tm_test :=
  <{ let processSum =
    (\x:Nat + Nat,
    case x of
    | inl n => n
    | inr n => (if0 n then $1 else $0)) in
    (processSum (inl Nat 5), processSum (inr Nat 5)) }>.

Example typechecks :
  <{ empty |-- tm_test \in Nat × Nat }>.
Proof. unfold tm_test. eauto 10. (* FILL IN HERE *) Admitted.

Example reduces :
  tm_test ->* <{ ($5, $0) }>.
Proof.
(*
  unfold tm_test. normalize.
*)
(* FILL IN HERE *) Admitted.

End Sumtest2.

```

Lists

```

Module ListTest.

(* let l = cons 5 (cons 6 (nil Nat)) in
  case l of
  nil => 0

```

```

| x::y => x*x *)

Definition tm_test :=
  <{ let l = ($5 :: $6 :: (nil Nat)) in
    case l of
    | nil => $0
    | x :: y => (x × x) }>.

Example typechecks :
  <{ empty |-- tm_test \in Nat }>.
Proof. unfold tm_test. eauto. (* FILL IN HERE *) Admitted.

Example reduces :
  tm_test ->* 25.
Proof.
  (*
    unfold tm_test. normalize.
  *)
  (* FILL IN HERE *) Admitted.

End ListTest.

```

fix

```

Module FixTest1.

Definition fact :=
  <{ fix
    (\f:Nat→Nat,
     \a:Nat,
     if_0 a then $1 else (a × (f (pred a)))) }>.

```

(Warning: you may be able to typecheck `fact` but still have some rules wrong!)

```

Example typechecks :
  <{ empty |-- fact \in Nat → Nat }>.
Proof. unfold fact. auto 10. (* FILL IN HERE *) Admitted.

Example reduces :
  <{ fact $4 }> ->* 24.
Proof.
  (*
    unfold fact. normalize.
  *)
  (* FILL IN HERE *) Admitted.

End FixTest1.

Module FixTest2.

```

```

Definition map :=
  <{ \g:Nat→Nat,
    fix
      (\f:(List Nat)→(List Nat),
       \l:List Nat,
       case l of
       | nil => nil Nat
       | x::l => ((g x)::(f l))) }>.

```

```

Example typechecks :
  <{ empty |-- map \in
    (Nat → Nat) → (List Nat) → (List Nat) }>.
Proof. unfold map. auto 10. (* FILL IN HERE *) Admitted.

Example reduces :
  <{ map (\a:Nat, succ a) ($1 :: $2 :: (nil Nat)) }>
  ->* <{ $2 :: $3 :: (nil Nat) }>.
Proof.
  (*
    unfold map. normalize.
  *)
  (* FILL IN HERE *) Admitted.

```

```

End FixTest2.

Module FixTest3.

Definition equal :=
  <{ fix
    (\eq:Nat→Nat→Nat,
     \m:Nat, \n:Nat,
     if0 m then (if0 n then $1 else $0)
     else (if0 n
           then $0
           else (eq (pred m) (pred n)))) }>.

Example typechecks :
  <{ empty |-- equal \in Nat → Nat → Nat }>.
Proof. unfold equal. auto 10. (* FILL IN HERE *) Admitted.

Example reduces :
  <{ equal $4 $4 }> --* 1.
Proof.
  (*
    unfold equal. normalize.
  *)
  (* FILL IN HERE *) Admitted.
  (* GRADE_THEOREM 0.25: reduces *)

Example reduces2 :
  <{ equal $4 $5 }> --* 0.
Proof.
  (*
    unfold equal. normalize.
  *)
  (* FILL IN HERE *) Admitted.
  (* GRADE_THEOREM 0.25: reduces2 *)

End FixTest3.

Module FixTest4.

Definition eotest :=
  <{ let evenodd =
    fix
      (\eo: (Nat → Nat) × (Nat → Nat),
       (\n:Nat, if0 n then $1 else (eo.snd (pred n)),
        \n:Nat, if0 n then $0 else (eo.fst (pred n)))) in
    let even = evenodd.fst in
    let odd = evenodd.snd in
    (even $3, even $4) }>.

Example typechecks :
  <{ empty |-- eotest \in Nat × Nat }>.
Proof. unfold eotest. eauto 30. (* FILL IN HERE *) Admitted.

Example reduces :
  eotest --* <{ ($0, $1) }>.
Proof.
  (*
    unfold eotest. eauto 10. normalize.
  *)
  (* FILL IN HERE *) Admitted.

End FixTest4.
End Examples.
□

```

Properties of Typing

The proofs of progress and preservation for this enriched system are essentially the same (though of course longer) as for the pure STLC.

Progress

Exercise: 3 stars, standard (STLCExtended.progress)

Complete the proof of progress.

Theorem: Suppose empty $\vdash t \in T$. Then either 1. t is a value, or 2. $t \rightarrow t'$ for some t' .

Proof: By induction on the given typing derivation.

```
Theorem progress : ∀ t T,  
  <{ empty |-- t \in T }> →  
  value t ∨ ∃ t', t → t'.
```

+

□

Weakening

The weakening claim and (automated) proof are exactly the same as for the original STLC. (We only need to increase the search depth of eauto to 7.)

```
Lemma weakening : ∀ Gamma Gamma' t T,  
  includedin Gamma Gamma' →  
  <{ Gamma |-- t \in T }> →  
  <{ Gamma' |-- t \in T }>.
```

Proof.

```
intros Gamma Gamma' t T H Ht.  
generalize dependent Gamma'.  
induction Ht; eauto 7 using includedin_update.
```

Qed.

```
Lemma weakening_empty : ∀ Gamma t T,  
  <{ empty |-- t \in T }> →  
  <{ Gamma |-- t \in T }>.
```

Proof.

```
intros Gamma t T.  
eapply weakening.  
discriminate.
```

Qed.

Substitution

Exercise: 2 stars, standard (STLCExtended.substitution_preserves_typing)

Complete the proof of substitution_preserves_typing.

```
Lemma substitution_preserves_typing : ∀ Gamma x U t v T,  
  <{ x ⊢ U ; Gamma |-- t \in T }> →  
  <{ empty |-- v \in U }> →  
  <{ Gamma |-- [x:=v]t \in T }>.
```

+

□

Preservation

Exercise: 3 stars, standard (STLCExtended.preservation)

Complete the proof of preservation.

```
Theorem preservation : ∀ t t' T,  
  <{ empty |-- t \in T }> →  
  t → t' →  
  <{ empty |-- t' \in T }>.
```

+

□

End `STLCExtended`.

(* 2026-01-07 13:33 *)

Index

This page has been generated by coqdoc